Review the assignment due date

Final Project

Team Number: Team 15

Team Name: Argus

Team Member Name	Email Address
Seth Lee	sethlee@seas.upenn.edu
Jerry Zhang	jerryjz@seas.upenn.edu
Thomas Liu	tholiu@seas.upenn.edu

GitHub Repository URL: https://github.com/upenn-embedded/final-project-s26-t15.git

GitHub Pages Website URL: [for final submission]*

Final Project Proposal

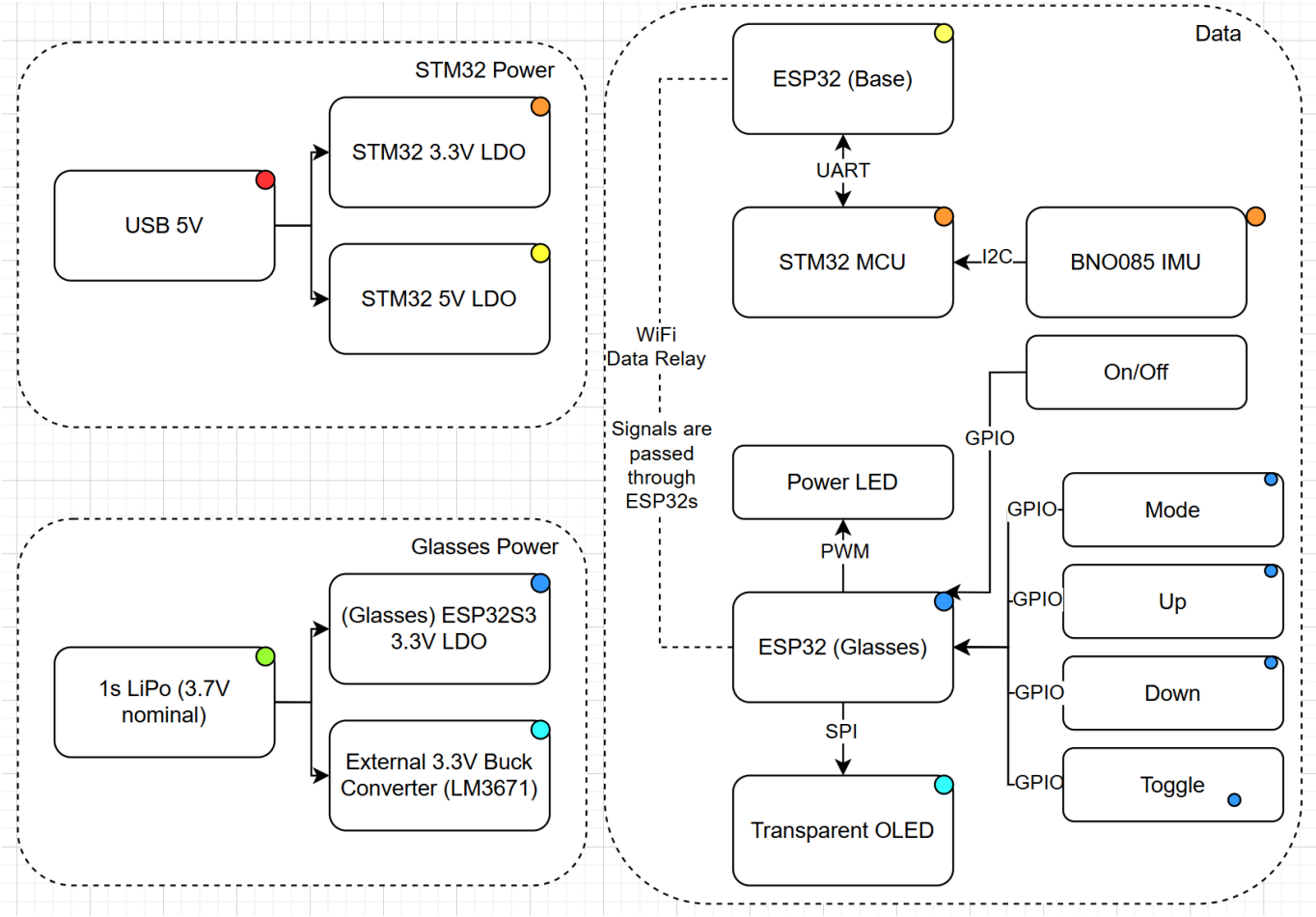
1. Abstract

Our final project (Argus) is a pair of augmented reality glasses that utilizes a HUD and sensor suite to collect and display movement information along with live timers that always keep the user up to date with their activities. Using a STM32 MCU and XIAO ESP32S3 processors, we can wirelessly transmit data from the glasses to our pocket. Featuring a transparent OLED for the HUD, 9-axis IMU for spatial tracking, and capacitive touch sensors, we combine portability with performance. The ESP32s will relay data wirelessly from the glasses to the STM32 in the user's pocket which serves as the compute hub.

2. Motivation

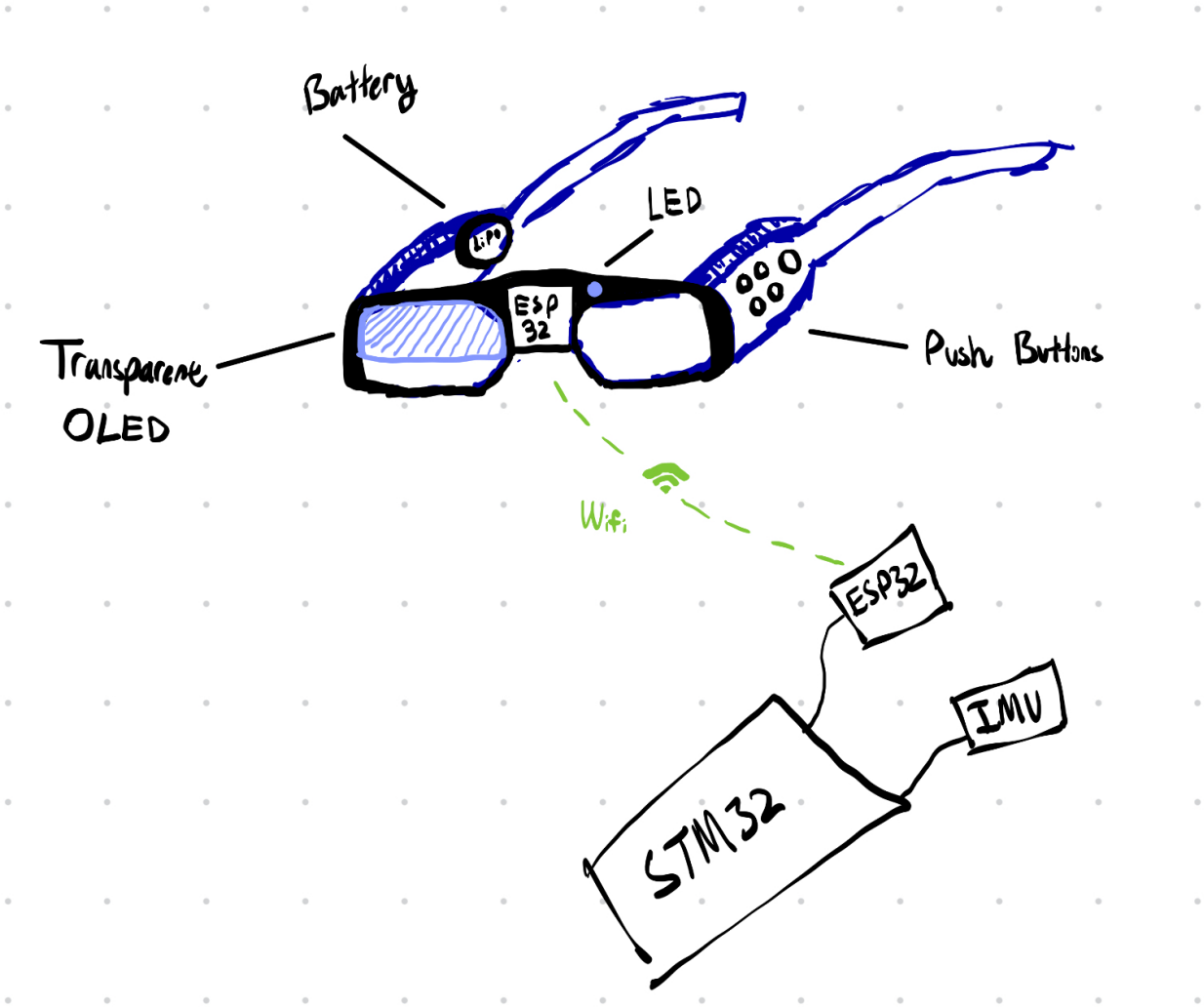
Argus helps bridge the gap between physical activity and real-time data accessibility without sacrificing situational awareness. While commercial AR glasses often prioritize complex media ecosystems and overlays, Argus focuses on lightweight, purpose-focused architecture that offloads heavy processing to a pocket-based STM32 to minimize weight. The sensors then provide a distraction-free, hands-free HUD for users. Instead of having to look at a watch or pull out a phone, users can find relevant telemetry in their line of sight. The emphasis on power efficiency and simplicity makes it perfect for tasks like checking the direction of travel while running or biking, where looking down could be unsafe. We hope that this project can provide a base for more advanced sensors and features.

3. System Block Diagram



4. Design Sketches

The frame of the glasses will be customized through 3D printing and laser cutting in order to house the hardware components.



5. Software Requirements Specification (SRS)

5.1 Definitions, Abbreviations

STM32 - STM32F411RE microcontroller on the NUCLEO-F411RE development board.

Base ESP32 - the ESP32 module connected to the STM32 via UART.

Glasses ESP32 - The ESP32 module mounted on the glasses frame. Receives data from the Base ESP32 and interacts with peripherals.

IMU - Inertial Measurement Unit. Accelerator/gyroscope sensor for step detection.

5.2 Functionality

ID	Description
SRS-01	The STM32 shall read the IMU accelerometer data over I2C at a minimum rate of 50 Hz to enable accurate step detection.
SRS-02	The step detection algorithm shall detect a walking step within 400ms of it occuring, with no more than 10% error over a 20-step walk test.

ID	Description
SRS-03	The display shall update its shown content within 500ms of a button press. This includes the full round-trip: glasses ESP32 → base ESP32 → STM32 → base ESP32 → glasses ESP32 → display.
SRS-04	The STM32 shall synchronize its RTC with NTP time via the base ESP32 over UART once at startup.
SRS-05	The display shall cycle through atleast three different modes: time view, step count view, and compass view.
SRS-06	The STM32 shall receive and process data from the base ESP32 over UART within 100ms of transmission.
SRS-07	The STM32 shall compute the compass direction from the magnetometer data from the IMU and display cardinal direction (N, NE, E, SE, S, SW, W, NW) accurate to within +/- 22.5 degrees after calibration.
SRS-08	The OLED display will communicate using SPI translated through UART wirelessly between the two ESP32S3 modules to maximize screen data bandwidth.

6. Hardware Requirements Specification (HRS)

6.1 Definitions, Abbreviations

See Section 5.1 for all definitions and abbreviations.

6.2 Functionality

ID	Description
HRS-01	The system shall use an IMU with atleast 3-axis accelerometer, 3-axis gyroscope, and 3-axis magnetometer capabilities, communicating over I2C, to provide motion data for step-count and compass data for direction.
HRS-02	The display shall be smaller enough to mount on glasses frames and support atleast two lines of text output.
HRS-03	Several push buttons shall be mounted on the glasses frame and be able to register a press within 100ms of acutuation to enable various functionalities such as toggling modes, changing parameters, and powering on and off.
HRS-04	Two ESP32 modules shall be able to maintain wireless connection with a round-trip latency of less than 500ms within 5m of distance.
HRS-05	The STM32 shall communicate with the base ESP32 over UART at a baud rate of atleast 9600 bps.
HRS-06	The glasses shall be powered by a battery capable of powering the glasses for atleast 15-30 minutes continuously.

7. Bill of Materials (BOM)

The major components needed are as follows: Nucleo STM32 F411RE Devboard, 2x XIAO ESP32S3 modules (wifi communication), DFR0934 monochrome transparent OLED (HUD), 9-DOF IMU (motion tracking), a LED to indicate power and debug features, external buck converter to isolate OLED from possibly noisy ESP32 LDO, 5 switches (mode, up, down, toggle, on/off), and a battery.

Component	Part Selection	Justification & Requirement Link
Main Processor	NUCLEO-F411RE	Master Controller: Handles the step-detection algorithm (SRS-02), RTC synchronization (SRS-04), and I2C sensor polling at 50 Hz (SRS-01). Selected for its ability to use interrupts and also industry standard usage.
Wireless Bridge	2x XIAO ESP32S3	Communication Relay: Facilitates the 500ms round-trip display update (SRS-03) and provides the wireless link required by HRS-04. Operates as a simple relay with no application code to satisfy system constraints.
Transparent HUD	DFR0934 OLED	Visual Interface: Meets HRS-02 for small-form-factor mounting and supports the multi-mode display requirements (Time, Steps, Compass) outlined in SRS-05.
Motion Tracking	9-DOF IMU (BNO085)	Navigation & Fitness: Provides the 3-axis Accel/Gyro/Mag data required by HRS-01. Enables compass accuracy within 22.5 degrees (SRS-07) and step detection (SRS-02).
Power Isolation	Buck Converter	System Stability: Ensures that WiFi bursts do not cause brownouts or electrical issues, allowing the battery to meet the 15 to 30 minute continuous run-time requirement (HRS-06).
User Input	5x Tactile Switches	Manual Control: Satisfies HRS-03 for registration within 100ms. Enables mode toggling (SRS-05) and system power management.
Power Source	1S LiPo Battery	Portable Battery: Selected to meet the weight requirements for glasses-mounting while satisfying the run-time specifications of HRS-06.
Status Indicator	Power LED	Visual Feedback: Provides an immediate hardware-level On state to confirm the success of power-on sequences defined in HRS-03.

https://docs.google.com/spreadsheets/d/1X19fjBgSUE6MPIu0bsC3vZN--DqyP1meB3pUoyBd7XM/edit?usp=sharing

8. Final Demo Goals

On Demo day, the smart glasses will be demonstrated by having someone wear them. The STM32 with its base ESP32 will have remain close by when the user is using the smart glasses to reduce latency and improve connectivity between the two ESP32 modules. Features will be tested by activating them using the side push buttons and moving around which will verify functionality of IMU features (cardinal direction and step counting) and the screen updating.

9. Sprint Planning

Milestone	Functionality Achieved	Distribution of Work

Milestone	Functionality Achieved	Distribution of Work
Sprint #1	1] Get individual components to connect to each other, i.e STM32 and ESP32. 2] Verify UART connection between STM32 and ESP32, wifi-connection, and confirm that STM32 can recieve button press. 3] Begin prototyping glasses frame + begin calibrating screen for close-up view. 4] synchronize RTC time with NTP time via wifi.	Seth - Firmware Jerry - Display + Firmware Thomas - Glasses Frame
Sprint #2	1] Get STM32 to receive IMU data over I2C and display steps / compass direction 2] Get display to show text/time. 3] Finalize power source for the glasses display + ESP32. 4] Finalize Glasses Frame and test it for proper fitting/balance.	Seth - Firmware Jerry - Display + Peripheral Integration Thomas - Glasses Frame
MVP Demo	1] Get steps algorithm and compass direction working. 2] Fully integrate ESP32 and display onto glasses frame. 3] Ensure that calibration + synchronizing of RTC time work as expected.	Seth - Firmware Jerry - Peripheral Integration + Calibration Thomas - Integration
Final Demo	Ensure that final product is fully functional and meets requirements.	

This is the end of the Project Proposal section. The remaining sections will be filled out based on the milestone schedule.

Sprint Review #1

Last week's progress

1. Finalized most of BOM and determined general design of prototype that we are now working towards.
2. Ordered parts for parts of the deisgn that we are more certain on
3. Planning design and build schedule.

Current state of project

1. Found several issues with current design mostly surrounding difficulty of writing I2C driver with our IMU.

-> Strange behavior listed in errata of F411 where clock stretching results in miscommunication between master and slave, which is problematic since the previous IMUs chosen (BNO085 or BNO055) both use it. Also discovered these devices don't contain built in step counter and I2C will be very difficult to write.

-> Switched to LSM6DS0 because it contains step counter and works over SPI. Since screen also uses SPI, it makes it convenient so we can write one driver that is also easier to write.

2. Button presses and interacting with the STM32F411 figured out

-> Basic interrupt or GPIO functionality

3. Calculated object distance (d0) constraints from a target virtual image distance (d1) and magnifcation (m0) constraint.

   d0 calculator (32)

$$d_0 = 24$$

0 32

$$f = 32$$

0 32

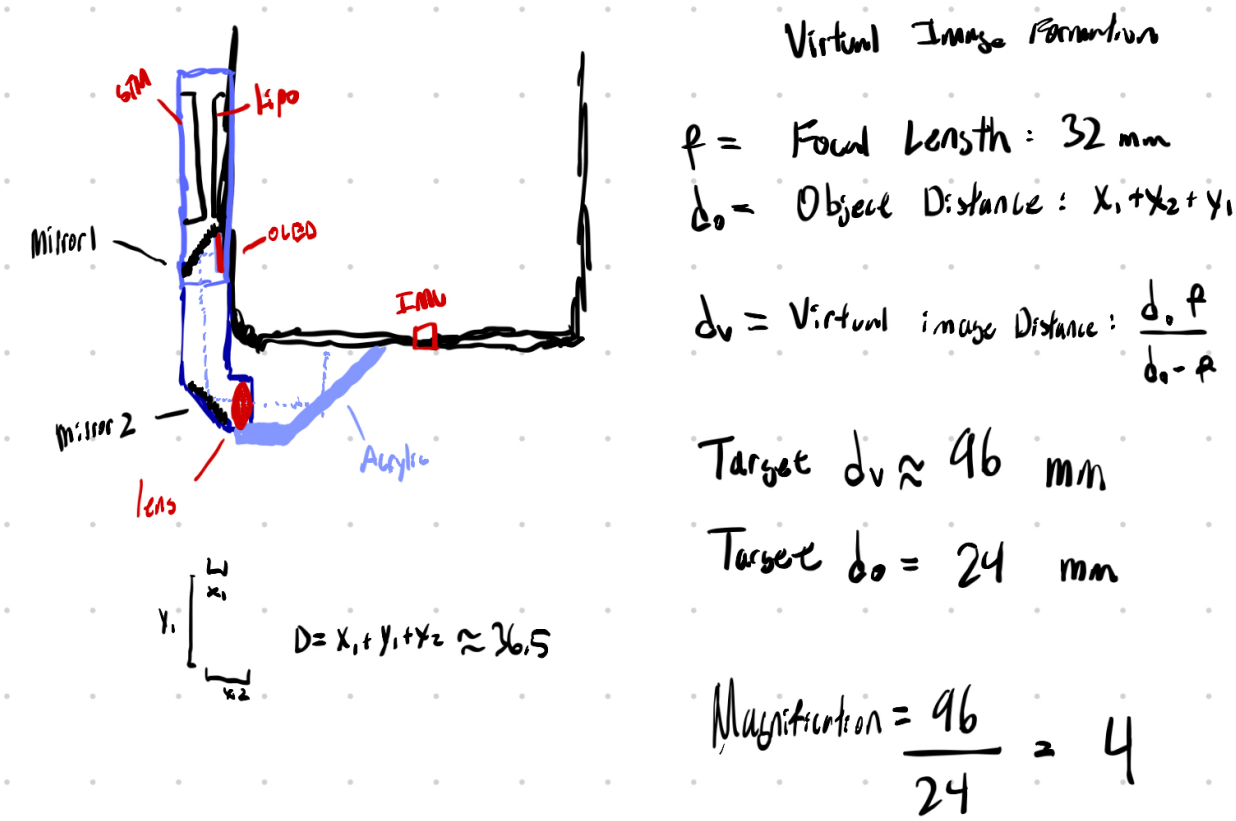
$$d_1 = \frac{d_0 \cdot f}{(d_0 - f)}$$

$$= -96$$

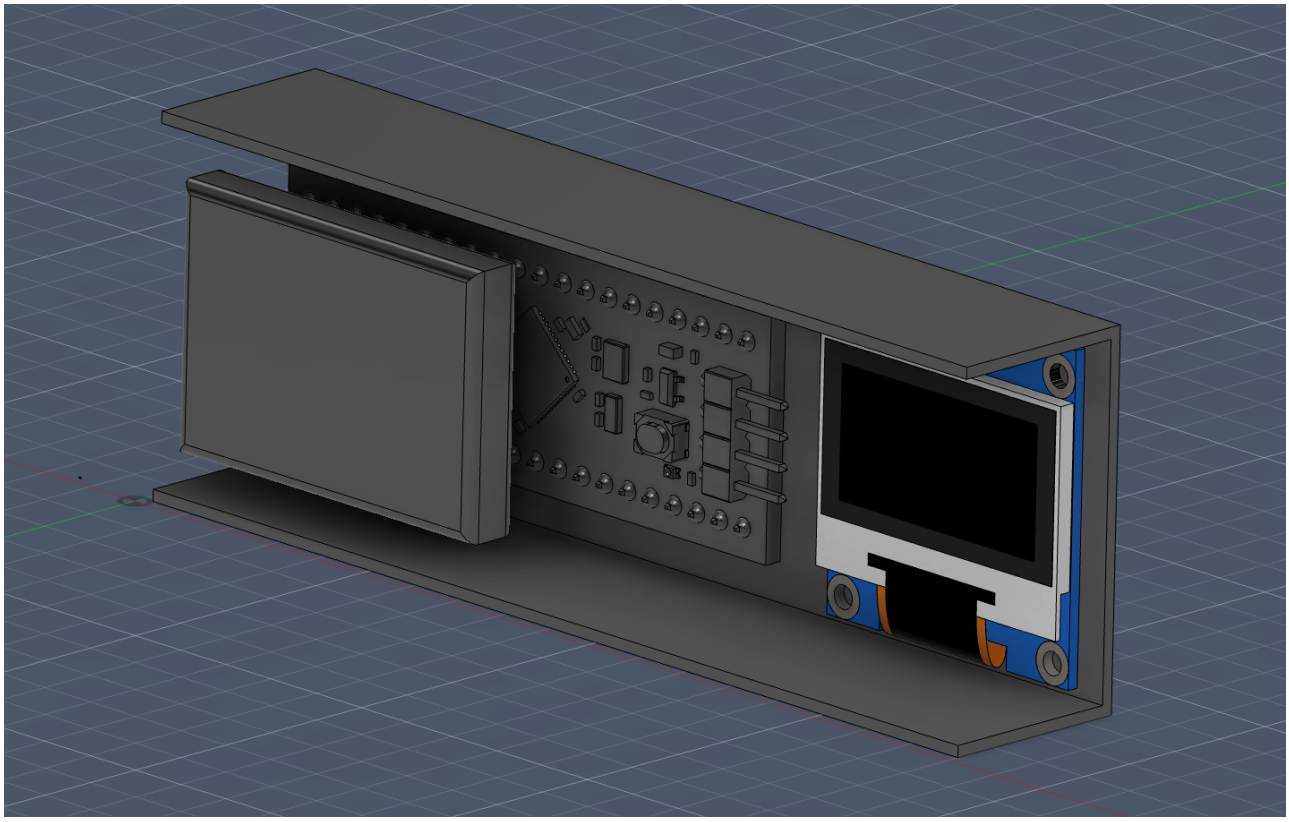
$$m_1 = \frac{-d_1}{d_0}$$

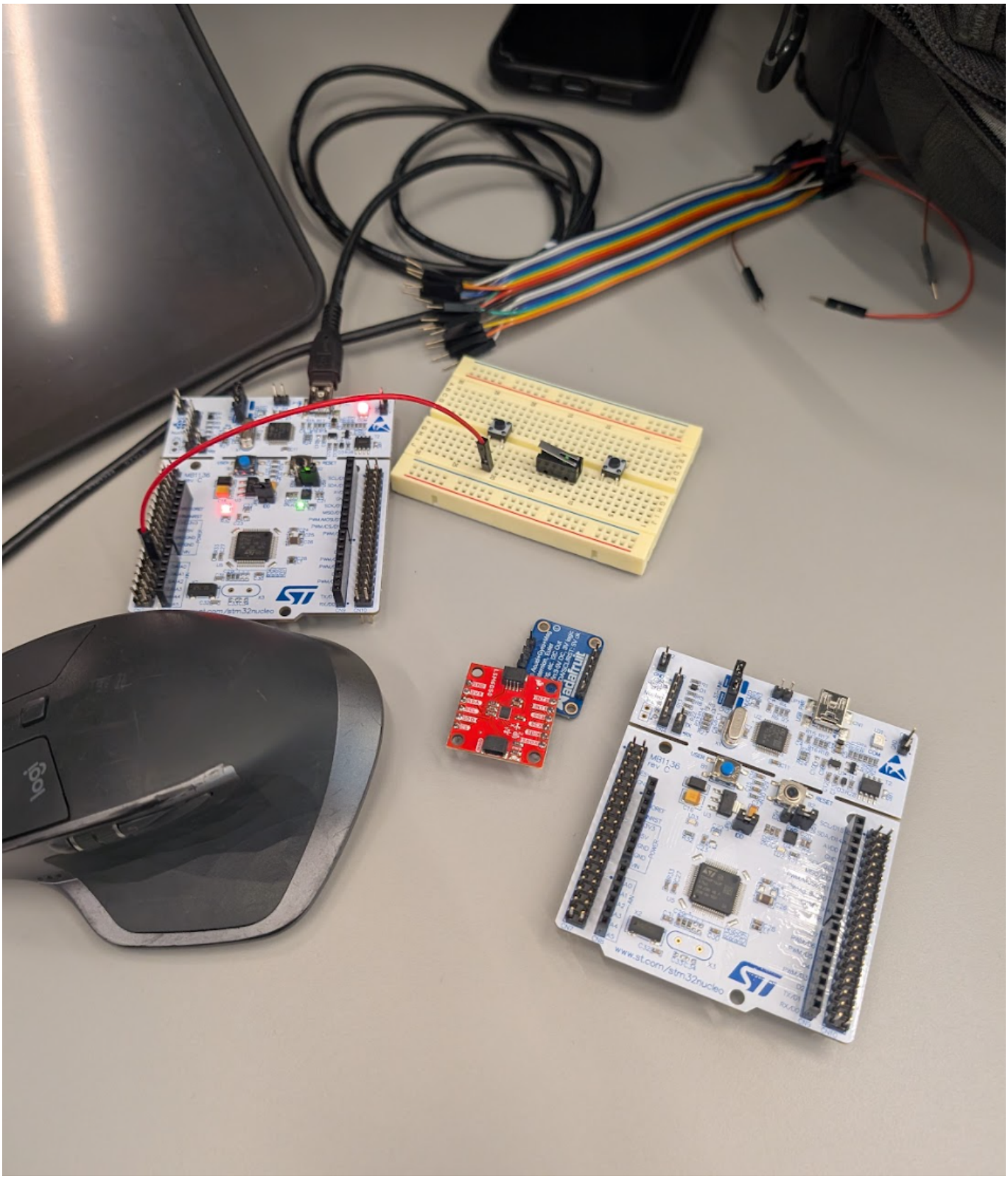
$$= 4$$

4. Designed the mechanical layout for the hardware and optics Determined methods to reduce d0 due to limited focal length, and reduce mount size by adding a secondary mirror.



5. Prototyped mounts to hold hardware so they can be clipped onto glasses.





Next week's plan

1. Finish full implementation of step counter (Seth)

-> Requires full functionality of SPI drivers

-> Get it to read out onto a serial output

-> Est. time: 1hr if SPI driver is working

2. (If screen arrives) Write library and implement basic functionality of screen (Jerry)

-> Allow for some basic interfacing with screen, determine how to set the screen color or edit pixels

-> Est. time: 2-3hrs

3. Write SPI driver (Jerry)

-> SPI driver has all necessary features for at least the IMU at this time, can initialize and operate independently without excessive debugging or compatibility issues with devices

-> Est. time: 3-6hrs

4. Write IMU driver (Seth)

-> Implement all major functions (get linear movement, step tracking, etc)

-> Est. time: 3-6hrs

5. CAD and print out a prototype (Thomas)

-> 3D-print a prototype (or gather a pair of real glass) and mount a dummy weight mimicking our device onto glasses and test weight distribution.

-> Est. time: 3-6hrs

Hopefully we also get some parts in. Work Distribution: Seth: Firmware and development of STM32 portions Jerry: Electrical hardware and writing SPI driver Thomas: Mechanical design and prototyping

In case anyone is not available to work on something, everyone is reading the STM32 textbook and is proficient in mechanical design so we can assist each other if extra work is needed.

Sprint Review #2

Last week's progress

Current state of project

Next week's plan

MVP Demo

Final Report

Don't forget to make the GitHub pages public website! If you’ve never made a GitHub pages website before, you can follow this webpage (though, substitute your final project repository for the GitHub username one in the quickstart guide): https://docs.github.com/en/pages/quickstart

1. Video

2. Images

3. Results

3.1 Software Requirements Specification (SRS) Results

ID	Description	Validation Outcome
SRS-01	The IMU 3-axis acceleration will be measured with 16-bit depth every 100 milliseconds +/-10 milliseconds.	Confirmed, logged output from the MCU is saved to "validation" folder in GitHub repository.

3.2 Hardware Requirements Specification (HRS) Results

ID	Description	Validation Outcome
HRS-01	A distance sensor shall be used for obstacle detection. The sensor shall detect obstacles at a maximum distance of at least 10 cm.	Confirmed, sensed obstacles up to 15cm. Video in "validation" folder, shows tape measure and logged output to terminal.

4. Conclusion

References